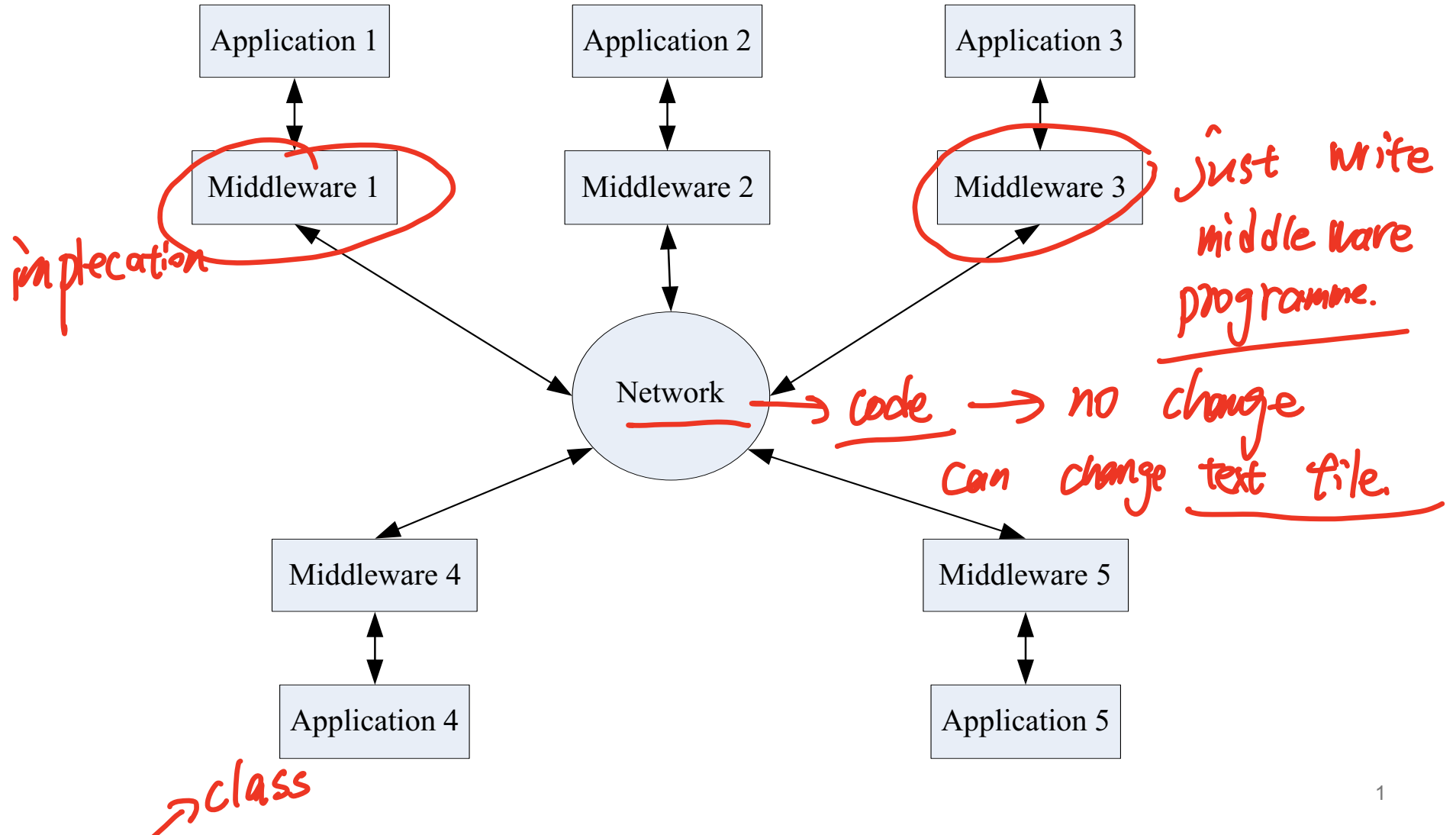




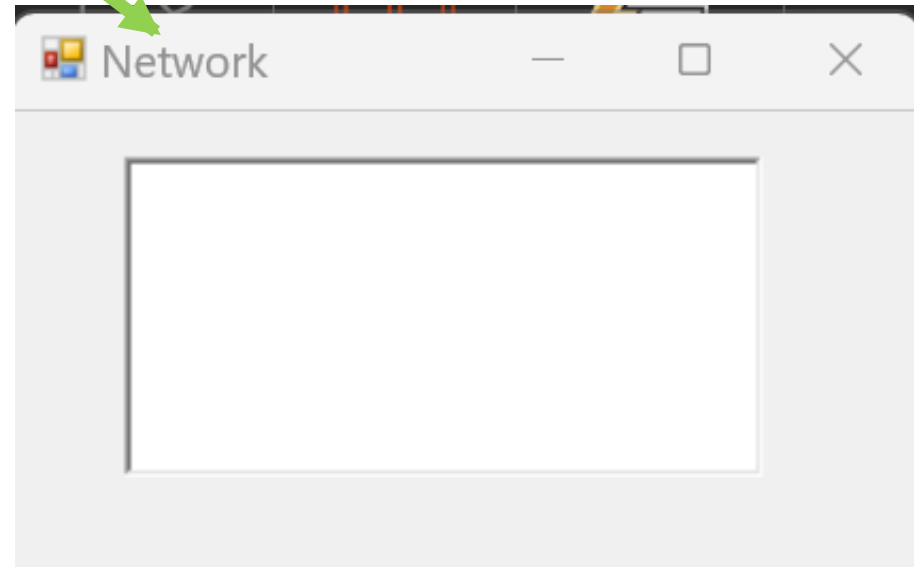
```
public Form1()  
{
```

```
    this.Text = "Network";  
    ...  
}
```

This is the network program.

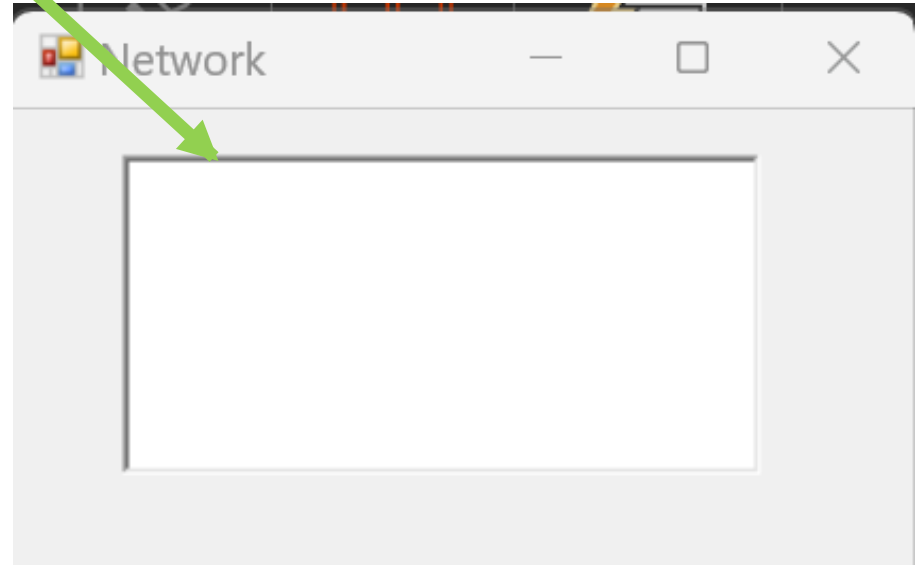
Form → window

- Set the title of the frame



```
public Form1()
{
    this.Text = "Network";
    richTextBox1 = new RichTextBox();
    ...
}
```

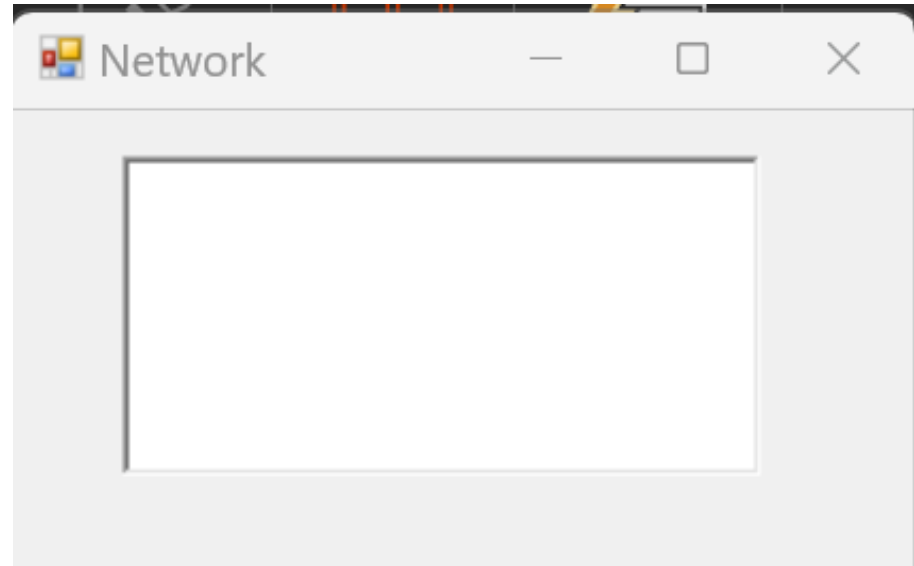
- Create a RichTextBox object



```
public Form1()
{
    this.Text = "Network";
    richTextBox1 = new RichTextBox();
    richTextBox1.Location = new System.Drawing.Point(35, 15);
    richTextBox1.Width = 200;
    richTextBox1.Height = 100;
    richTextBox1.Multiline = true;
    richTextBox1.ScrollBars = RichTextBoxScrollBars.Vertical;

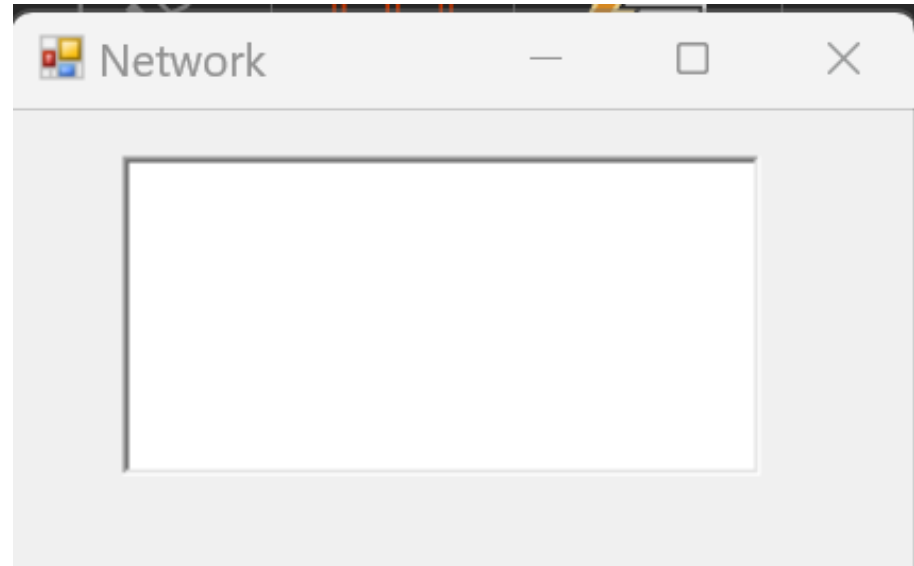
    ...
}
```

- Set the location of the RichTextBox
- Set the size of the box
- Set properties of the box



```
public Form1()  
{  
    this.Text = "Network";  
    richTextBox1 = new RichTextBox();  
    richTextBox1.Location = new System.Drawing.Point(35, 15);  
    richTextBox1.Width = 200;  
    richTextBox1.Height = 100;  
    richTextBox1.Multiline = true;  
    richTextBox1.ScrollBars = RichTextBoxScrollBars.Vertical;  
  
    Controls.Add(richTextBox1);  
  
    ...  
}
```

- Add RichTextBox object to the window



```
public Form1()
{
```

```
...
```

```
listener = new TcpListener(IPAddress.Any, 8081);
listener.Start();
ListenForClientsAsync();
```

```
}
```

TCP 协议.

parameter, 参数

port number
端口号.

accept any message

call method

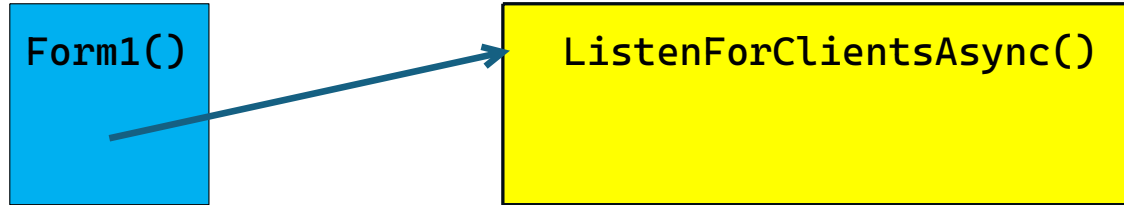
process message.

- private TcpListener listener;
- TcpListener listens for incoming TCP connection requests.
- IPAddress.Any means that the listener will listen on all network interfaces available on the machine
- 8081 is the port number
- After the object is created, it starts working

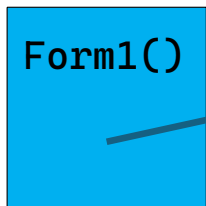
```
public Form1()
{
    ...

    listener = new TcpListener(IPAddress.Any, 8081);
    listener.Start();
    ListenForClientsAsync();

    ...
}
```



/ receive message.



facing block → ← return caller. → continue ...
use other way.

异步操作

```
private async void ListenForClientsAsync()
```

```
{
```

```
while (true)
```

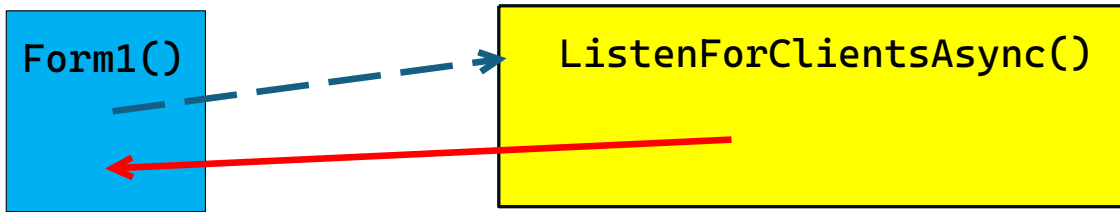
```
{
```

```
TcpClient client = await listener.AcceptTcpClientAsync();  
ReadMessageAsync(client);
```

```
}
```

```
}
```

- **AcceptTcpClientAsync** waits for incoming TCP connection
- **await** blocks the execution of the method and returns to the caller, i.e., the caller resumes its execution



```
public Form1()
{
```

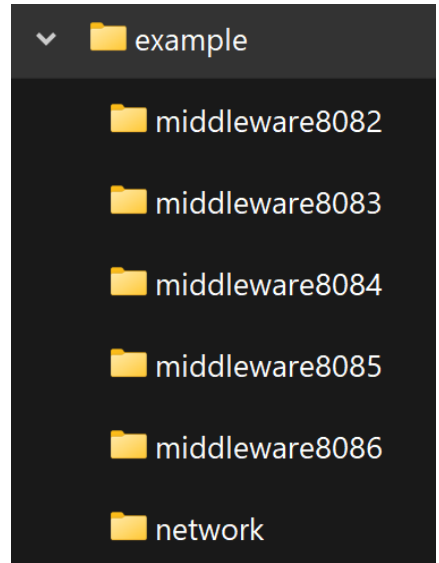
...

```
listener = new TcpListener(IPAddress.Any, 8081);
listener.Start();
ListenForClientsAsync();
```

block *Continue*
↓

```
string currentDirectory = Directory.GetCurrentDirectory();
string filePath = Path.Combine(currentDirectory, "network", "delays.txt");
delayLines = File.ReadAllLines(filePath);
```

tell file path
应用程序路径



- Use relative path
 - We start under the 'example' directory
- "delays.txt" records the delays for sending message to a middleware

```

public Form1()
{
    ...

    string currentDirectory = Directory.GetCurrentDirectory();
    string filePath = Path.Combine(currentDirectory, "network", "delays.txt");
    delayLines = File.ReadAllLines(filePath);
}

```

数组

• private string[] delayLines;

Contents of delays.txt

```

1,8000,100,200,300
8000,100,1000,500,5000
1,8000,100,200,300
8000,100,1000,500,5000

```

middleware

m1 m2 m3 m4 m5

1 , 100 , 30 , 40 , 50

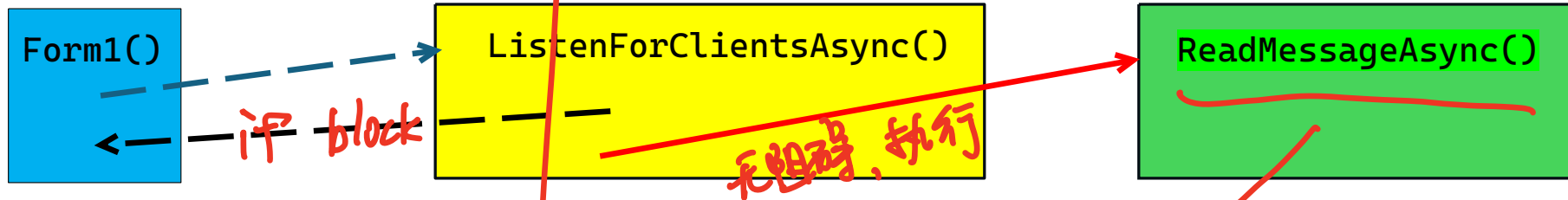


*Time wait to responding middleware.
(hold the message.)*

若无阻塞.

```
private async void ListenForClientsAsync()  
{  
    while (true)  
    {  
        TcpClient client = await listener.AcceptTcpClientAsync();  
        ReadMessageAsync(client);  
    }  
}
```

detail no wait



```
private async void ReadMessageAsync(TcpClient client)
```

acquire data, be sent

```
{  
    byte[] buffer = new byte[1024];
```

name.

```
    await client.GetStream().ReadAsync(buffer, 0, buffer.Length);
```

```
    string message = Encoding.UTF8.GetString(buffer).Trim('\0');
```

```
    richTextBox1.AppendText($"Receive message {message}\n");
```

```
    SendMessagesAsync(message);
```

```
}
```

send to middle ware.

end with
null byte
delete null
byte

- Buffer for storing incoming message. For the async method, even if multiple callers call the method concurrently, each caller has their own copy of buffer.
- ReadAsync get message from the TCP socket
- Trim('\0') The buffer is initialised to 0 and 0s are not part of the incoming message.
- richTextBox1.AppendText show message in the text box

net program

二进制
转换

```
private async void ListenForClientsAsync()
{
    while (true)
    {
        TcpClient client = await listener.AcceptTcpClientAsync();
        ReadMessageAsync(client);
    }
}
```

waiting
message

block.

ReadMessageAsync(client);

...

ReadMessageAsync(client);

```
private async void ListenForClientsAsync()
{
    while (true)
    {
        TcpClient client = await listener.AcceptTcpClientAsync();
        ReadMessageAsync(client);
    }
}
```

ReadMessageAsync(client);

SendMessageAsync(message);

ReadMessageAsync(client);

SendMessageAsync(message);

...
各个消息一起发
↓
异步

← 多个重复执行 →

```
private async void ReadMessageAsync(TcpClient client)
{
    ...
    SendMessageAsync(message);
}
```

用lock来保证每个消息只读一次

```

private async void SendMessagesAsync(string message)
{
    string[] delays;
    lock (lockObject)
    {
        delays = delayLines[currentLine++].Split(',');
    }

    int[] intDelays = delays.Select(x=>int.Parse(x)).ToArray();
    ...
}

```

一个不同属性，后续若锁，等待
直到上一个结束解锁
↓
holding queue

check whether locked.
if locked, 等待.
unlocked, executed. 递增

shared
Do, 分别

类型变为
string

string 转 int.

- delays holds the values in one line of file delays.txt
- private object lockObject = new object();
- private int currentLine = 0; 行数
- Select(x=>int.Parse(x)) converts each element in delays to an integer

```
private async void SendMessagesAsync(string message)
{
    string[] delays;
    lock (lockObject)
    {
        delays = delayLines[currentLine++].Split(',');
    }
    int[] intDelays = delays.Select(x=>int.Parse(x)).ToArray();
    int[] sortedPorts = new int[5];
    Array.Copy(ports, sortedPorts, 5);
}
```

from *to*

- `private int[] ports = { 8082, 8083, 8084, 8085, 8086 };`
- `ports` records the ports on which the middleware waiting for TCP connections
- `Array.Copy(ports, sortedPorts, 5)` copies the 5 elements from `ports` to `sortedPorts`

```
private async void SendMessagesAsync(string message)
{
    string[] delays = delayLines[current...];
    int[] intDelays = delays.Select(x=>x...).ToArray();
    int[] sortedPorts = new int[5];
    Array.Copy(ports, sortedPorts, 5);

    Array.Sort(intDelays, sortedPorts);

    ...
}
```

example

index bind

↓		↓	↓	
0	(100	8082)	10	8086
1	(20	8083)	20	8083
2	(30	8084)	30	8084
3	(40	8085)	40	8085
4	(10	<u>8086</u>)	100	8082

↑
key

排序

- **Array.Sort** method to sort two arrays **simultaneously** based on the elements of **intDelays**.
- **intDelays**: This is the array whose elements will be used as **keys for sorting**. The elements in **intDelays** will determine the order in which corresponding elements in **sortedPorts** are sorted.
- **sortedPorts**: The elements in **sortedPorts** will be **rearranged** based on the sorting order **determined by the intDelays** array.

```

private async void SendMessagesAsync(string message)
{
    ...
    Array.Sort(intDelays, sortedPorts);
    for (int i = 0; i < 5; i++) 重复5次
    {
        if (i == 0)
            await Task.Delay(intDelays[i]);
        else
            await Task.Delay(intDelays[i]-intDelays[i-1]);
    }
}

```

send message to
first middle ware
first middle
ware.

- Send message to each middleware
- For the first middleware, wait for `intDelays[i]`
- For others, wait for `intDelays[i]-intDelays[i-1]`

```
private async void SendMessagesAsync(string message)
{
    ...
    for (int i = 0; i < 5; i++)
    {
        if (i == 0)
            await Task.Delay(intDelays[i]);
        else
            await Task.Delay(intDelays[i]-intDelays[i-1]);
        Set Connection TcpClient client = new TcpClient("localhost", sortedPorts[i]);
    }
}
```

name of machine.

- **TcpClient** class represents a client that connects to a TCP server

```
private async void SendMessagesAsync(string message)
```

```
{
```

```
    ...  
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        if (i == 0)
```

```
            await Task.Delay(intDelays[i]);
```

```
        else
```

```
            await Task.Delay(intDelays[i]-intDelays[i-1]);
```

```
            TcpClient client = new TcpClient("localhost", sortedPorts[i]);
```

```
            byte[] data = Encoding.UTF8.GetBytes(message);
```

```
            await client.GetStream().WriteAsync(data, 0, data.Length);
```

```
    }
```

delay

10
20 } 10.
30 } 10
40 } 10
50 } 10

how to
Send message
to network
}

↑

string 改成 二进程 发送的 middle ware.
index number of byte.

- Encoding.UTF8.GetBytes(message): Encode the message
- WriteAsync sends the message to the TCP socket

net program

```
public Form1()
```

```
{
```

```
    this.Text = "Middleware8082";
```

```
    button1 = new Button();
```

```
    button1.Size = new System.Drawing.Size(100, 50);
```

```
    button1.Location = new System.Drawing.Point(35, 15);
```

```
    button1.Text = "Send Message";
```

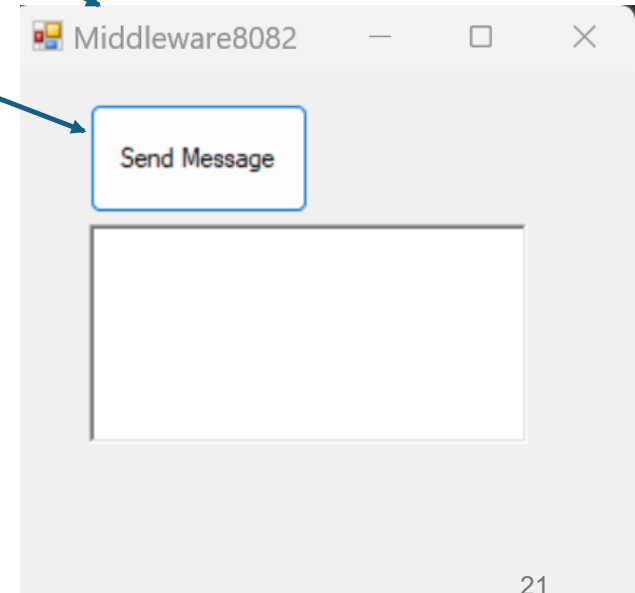
```
    button1.Click += new EventHandler(button1_Click);
```

```
}
```

... register a method to

This is the middleware program

- Register method button1_Click with event button1.Click
- When the button is clicked, the method will be executed.



```
public Form1()
{
    ...
    richTextBox1 = new RichTextBox();
    richTextBox1.Location = new System.Drawing.Point(35, 70);
    richTextBox1.Width = 200;
    richTextBox1.Height = 100;
    richTextBox1.Multiline = true;
    richTextBox1.ScrollBars = RichTextBoxScrollBars.Vertical;

    Controls.Add(button1);
    Controls.Add(richTextBox1);
}
```

加入 window

- Create the text box and set the properties.
- Add the text box and the button to the window



```
public Form1()
{
    ...
    Controls.Add(button1);
    Controls.Add(richTextBox1);

    listener = new TcpListener(IPAddress.Any, 8082);
    listener.Start();
    ListenForClientsAsync();
}
```

↓
similar to net programmer ...

- `private TcpListener listener;`
- `TcpListener` listens for incoming TCP connection requests.
- `ListenForClientsAsync` is the same as the one for network program.

send message from middle aware to network.

```
private async void button1_Click(object sender, EventArgs e)  
{
```

```
    TcpClient client = new TcpClient("localhost", 8081);
```

```
    byte[] data = Encoding.UTF8.GetBytes("from 8082");
```

```
    await client.GetStream().WriteAsync(data, 0, data.Length);
```

Send
message }

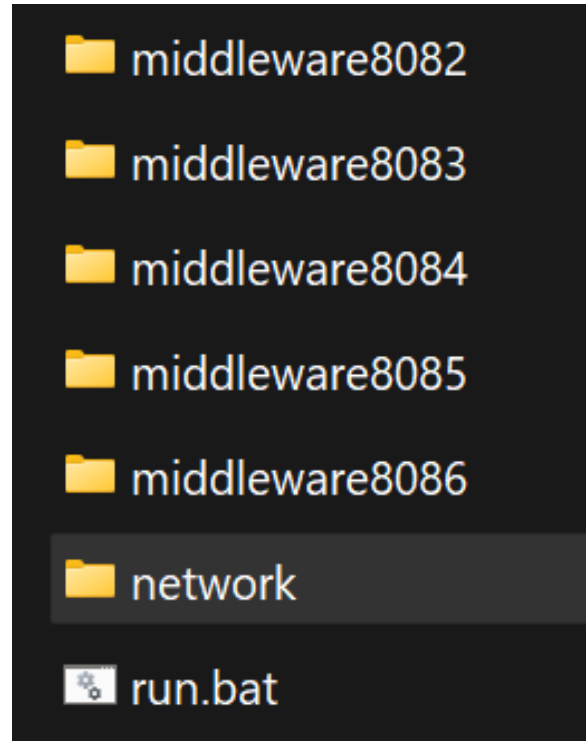
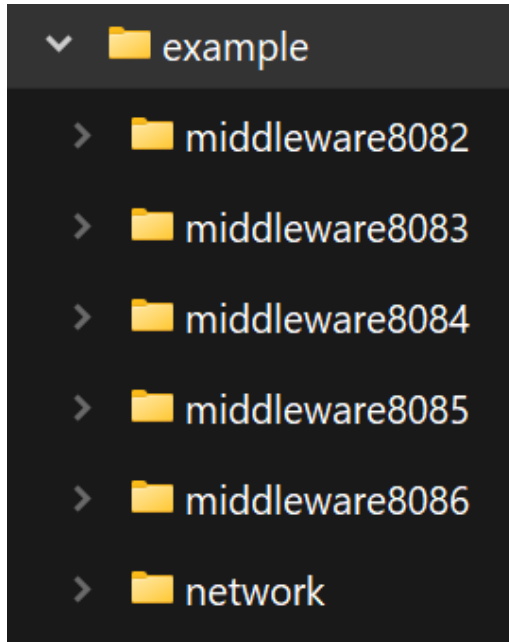
method.

- Connect to the network
- Encode the message
- Send the message through the TCP socket

- Compile program in Visual Studio 2022 Developer Command Prompt

→
Commoned.

- `csc /out:Network\Network.exe Network\Network.cs`





run.bat

run.bat



File Edit View



```
csc /out:network\network.exe network\network.cs
csc /out:middleware8082\middleware8082.exe middleware8082\middleware8082.cs
csc /out:middleware8083\middleware8083.exe middleware8083\middleware8083.cs
csc /out:middleware8084\middleware8084.exe middleware8084\middleware8084.cs
csc /out:middleware8085\middleware8085.exe middleware8085\middleware8085.cs
csc /out:middleware8086\middleware8086.exe middleware8086\middleware8086.cs
start .\network\network.exe
start middleware8082\middleware8082.exe
start middleware8083\middleware8083.exe
start middleware8084\middleware8084.exe
start middleware8085\middleware8085.exe
start middleware8086\middleware8086.exe
```

I